Universidad de los Andes Facultad de Ingeniería	**Ingeniería de Sistemas y Computación** **ISIS-1611: Inteligencia artificial** Semestre: 2026-10 Créditos: 3	Accredited ABET Engineering Accreditation Commission

Taller 2: CSP y Búsqueda Adversaria

Grupo 8

Por: Karen Fuentes (202122467), Sofia Sarasty (202511871), Juan Pablo Camacho Peña (202110977), Tatiana Rosales (202212897)

Distribución del trabajo

Integrante	Sección	Algoritmos
Juan Pablo Camacho Peña	Punto 1	Backtracking básico + Forward Checking
Sofia Sarasty	Punto 2	AC-3 + MRV/LCV (Bono)
Tatiana Rosales	Punto 3	Función de evaluación + MinimaxAgent
Karen Fuentes	Punto 4	ExpectimaxAgent + AlphaBetaAgent (Bono)

<u>Punto 1 — CSP: Backtracking y Forward Checking</u>

(Juan Pablo Camacho Peña)

Formulación del problema como CSP

El problema de asignación de drones se modela como una tripla (X, D, C):

- **Variables (X):** X = {X_1, X_2, …, X_m}, una variable X_i por cada punto de entrega E_i. Cada X_i pregunta: '¿qué dron realiza la entrega E_i?'
- **Dominios (D):** D(X_i) = {$drone_1$, $drone_2$, …, drone_n} para todo i. Inicialmente cualquier dron puede atender cualquier entrega.
- **Restricciones (C):**
  - Capacidad: Σ weight(X_i) ≤ capacity(drone_k) para cada drone_k asignado.
  - Batería: costo_total_ruta(drone_k) ≤ battery(drone_k). El costo se calcula con Dijkstra considerando el terreno.
  - Ventana de tiempo: t_early(X_i) ≤ t_arrival(X_i) ≤ t_late(X_i). El tiempo de llegada depende de las entregas previas del mismo dron.

1 Backtracking Básico

Complejidad en Tiempo: O(n^m), donde n es el número de drones y m el número de entregas. El árbol tiene n^m hojas posibles.

Complejidad en Espacio: O(m), proporcional a la profundidad de la recursión (número de entregas). Solo se guarda la pila de asignaciones activas.

¿Es completo?: Sí. Explora sistemáticamente todas las asignaciones posibles y garantiza encontrar una solución si existe.

¿Encuentra la solución óptima?: No aplica directamente; el CSP busca satisfacción, no optimización. La primera solución encontrada es válida pero no necesariamente la de menor costo.

El algoritmo selecciona la primera variable no asignada, prueba cada valor de su dominio en orden, verifica consistencia con is_consistent(), y retrocede si ningún valor es válido. Sin optimizaciones, puede explorar millones de combinaciones en problemas grandes.

Resultados en layouts CSP seleccionados:

Layout	Variables	Solución	Backtracking requerido
twin_bases	2	Sí	Mínimo — 2 entregas, restricciones suaves
tight_battery	2	Sí	Moderado — batería justa fuerza asignación específica
tight_windows	3	Sí	Alto — ventanas muy ajustadas, muchas combinaciones inválidas

big_fleet	5	Sí	Alto — 4^5 = 1024 combinaciones posibles
detour_required	1	Sí	Ninguno — solo 1 entrega, restricción de batería con desvío

2 Backtracking con Forward Checking (FC)

Complejidad en Tiempo: O(n^m) en el peor caso, pero en la práctica significativamente menor gracias a la poda temprana de dominios vacíos.

Complejidad en Espacio: O(m · n) — además de la pila de recursión, se guardan copias de los dominios de los vecinos para restaurarlos en backtrack.

¿Es completo?: Sí. FC solo poda ramas inviables y nunca elimina soluciones válidas.

¿Cuándo mejora sobre backtracking básico?: Siempre que la asignación actual reduzca el dominio de los vecinos. El beneficio es mayor en layouts con restricciones ajustadas (tight_windows, tight_battery, staggered_time).

Después de asignar var = value, FC elimina de los dominios de cada vecino no asignado los valores que ya no son consistentes con la asignación actual. Si algún dominio queda vacío, retrocede inmediatamente sin explorar esa rama.

Diferencia práctica con backtracking básico — FC detecta el fallo en este ejemplo:

Si drone1 (capacidad=5) ya tiene entrega de peso=3 asignada, y se asigna otra de peso=4 → FC detecta inmediatamente que ningún valor para la entrega restante es válido para drone1, y retrocede sin explorar más. El backtracking básico no detectaría esto hasta intentar asignar la siguiente entrega.

Limitación: FC solo verifica consistencia con los vecinos directos del nodo asignado. No propaga transitivamente las restricciones (eso lo hace AC-3).

<u>Punto 2 — CSP: AC-3 y MRV+LCV (Bono)</u>
(Sofia Sarasty)

3 Backtracking con AC-3

Complejidad en Tiempo: AC-3 global: $O(n^2 \cdot d^3)$ donde d = tamaño máximo del dominio. Por llamada: $O(\text{arcos} \times d^2)$. En la práctica mucho mejor que backtracking + FC.

Complejidad en Espacio: $O(m \cdot n)$ para las copias de dominios + O(arcos) para la cola de AC-3.

¿Es completo?: Sí. AC-3 solo elimina valores sin soporte; nunca elimina una solución válida.

Principio de arco-consistencia: una restricción (X_i, X_j) es arco-consistente si para todo valor $v \in D(X_i)$ existe al menos un valor $w \in D(X_j)$ tal que la asignación $X_i=v$, $X_j=w$ no viola ninguna restricción.

Algoritmo AC-3: mantiene una cola de arcos pendientes. Para cada arco (X_i, X_j), llama a revise(X_i, X_j) que elimina de $D(X_i)$ los valores sin soporte en $D(X_j)$. Si el dominio cambia, re-encola todos los arcos hacia X_i. Si algún dominio queda vacío, hay inconsistencia global.

Ventaja sobre FC: FC solo verifica consistencia en los vecinos del nodo recién asignado. AC-3 propaga la restricción transitivamente por toda la red, eliminando valores inviables incluso en variables lejanas.

Caso de uso ideal: layouts como detour_required o storm_corridor donde la propagación inicial puede determinar dominios unitarios sin necesidad de explorar el árbol.

Resultados obtenidos con AC-3:

Mapa	Cantidad Drones	Cantidad Deliveries	Tiempo en segundos
maze_csp	2	2	0,0001
battery_edge	1	1	0
fog_valley	2	1	0
big_fleet	4	5	0,0018
triple_fleet	3	3	0,0004

4 Backtracking con MRV + LCV (Bono)

Complejidad en Tiempo: La selección MRV añade O(m) por nivel; LCV añade O(n · m) por variable.

Complejidad en Espacio: O(n) donde n es el conteo total de valores para cada variable.

MRV (Minimum Remaining Values): Selecciona la variable con menos valores legales en su dominio actual. La intuición es 'resolver primero lo más difícil': si una variable ya tiene pocas opciones, detectar si lleva a fallo o éxito antes reduce el árbol de búsqueda. En caso de empate, se usa la heurística de grado: se prefiere la variable con más vecinos no asignados.

LCV (Least Constraining Value): Ordena los valores de una variable de menor a mayor número de conflictos que generarían en los dominios de los vecinos. Se intentan primero los valores que preservan más opciones para las variables restantes, maximizando las posibilidades de encontrar una solución sin backtracking.

Resultados obtenidos con MRV + LCV:

Mapa	Cantidad Drones	Cantidad Deliveries	Tiempo de ejecución en segundos
maze_csp	2	2	0,0001
battery_edge	1	1	0
fog_valley	2	1	0,0001
big_fleet	4	5	0,0011
triple_fleet	3	3	0,0002

Comparación de algoritmos CSP:

Algoritmo	Detección temprana	Nodos explorados	Overhead por nodo
Backtracking básico	Ninguna	Alto	Bajo
Backtracking + FC	Dominio vacío inmediato	Medio	Bajo-Medio
Backtracking + AC-3	Propagación transitiva	Bajo	Medio
MRV + LCV + FC	Dominio vacío + orden	Muy bajo	Medio

<u>Punto 3 — Búsqueda Adversaria: Función de Evaluación + Minimax</u>
(Leidy Tatiana Rosales Pulido)

Función de Evaluación f(s)

La función de evaluación es necesaria porque el árbol de juego completo es computacionalmente inabordable. Con profundidad limitada d, los estados no terminales en el nivel d se evalúan con f(s) para estimar su valor sin explorar más profundamente. El rango es [-1000, +1000], alineado con los estados terminales del juego.

La función combina cinco señales:

- **(a) Distancia BFS al delivery más cercano:** + 20/(distancia+1)
  Motiva al dron a acercarse a entregas pendientes. Se utiliza la distancia BFS para considerar obstáculos del mapa y obtener la distancia real de navegación. La función inversa hace que el beneficio sea mayor cuando el dron está cerca del objetivo.
- **(b) Distancia cazador → dron (terreno normal):** -30/(distancia+1)
  Penaliza estados en los que los cazadores están cerca del dron. A menor distancia, mayor penalización, incentivando al dron a alejarse de los cazadores.
- **(c) Entregas pendientes:** −10×(número de entregas restantes)
  Se penaliza tener muchas entregas pendientes para incentivar completar los objetivos lo antes posible.
- **(d) Puntuación actual:** + score
  Se incorpora la puntuación acumulada del estado para reflejar el progreso real del juego y favorecer estados donde el dron ya ha completado entregas.

Decisión de diseño — balance entre progreso y seguridad:
La función busca equilibrar dos objetivos principales: avanzar hacia las entregas y evitar a los cazadores. Las funciones inversas de distancia permiten que los estados cercanos a un objetivo o a un peligro tengan mayor impacto en la evaluación.

Limitación identificada:
La función no penaliza explícitamente revisitar posiciones del mapa. En layouts con pocos caminos posibles puede ocurrir oscilación entre dos celdas cuando dos estados reciben valores de evaluación similares.

5 MinimaxAgent

Complejidad en Tiempo: O(b^(d×n)) donde b = factor de ramificación, d = profundidad, n = número de agentes.

Complejidad en Espacio: O(d×n) — solo se almacena la rama activa de la búsqueda.

¿Es completo (con profundidad finita)?: Sí, para el subárbol de profundidad d.

¿Es óptimo?: Sí, frente a un oponente perfectamente adversarial. No frente a cazadores que actúan aleatoriamente (para eso está Expectimax).

Principio: el dron (MAX) elige la acción que maximiza el valor del estado resultante, asumiendo que cada cazador (MIN) elegirá la acción que lo minimiza. La profundidad d se decrementa solo cuando todos los agentes han completado su turno (una ronda completa).

Resultados en layouts adversariales seleccionados (p=0, depth=1):

Layout	Resultado	Observación
tiny_hunt	Victoria	Árbol pequeño, solución óptima garantizada
small_hunt	Victoria	Dron esquiva cazador y completa 2 entregas
contested_territory	Variable	Cazadores cercanos desde inicio — función de eval crítica
jungle_chase	Variable	Dos cazadores; depth=1 insuficiente, depth=2 mejora
swamp_run	Victoria	Cazadores bloqueados por niebla, función detecta ventaja
maze_hunt	Oscilación	El dron alterna entre dos posiciones debido a valores similares en la evaluación

Limitación de Minimax: Minimax asume que los cazadores siempre actúan de forma óptima para minimizar el valor del dron. Si los cazadores se comportan de forma aleatoria (p>0), esta suposición puede hacer que el dron adopte estrategias excesivamente conservadoras, evitando rutas que en la práctica serían seguras. En estos casos, algoritmos como Expectimax pueden modelar mejor el comportamiento probabilístico de los oponentes.

<u>Punto 4 — Búsqueda Adversaria: Expectimax + AlphaBeta (Bono)</u>
(Karen Fuentes)

6 ExpectimaxAgent

Complejidad en Tiempo: O(b^(d×n)) — igual que Minimax. No hay poda posible en nodos de azar.

Complejidad en Espacio: O(d×n) — solo la pila de recursión activa.

¿Es completo?: Sí, para el subárbol de profundidad d.

¿Es óptimo?: Óptimo en valor esperado si el modelo de probabilidades es correcto. Si p es incorrecto, puede tomar decisiones subóptimas.

Cada cazador tiene un nodo de azar (chance node) en lugar de un nodo MIN. Con el parámetro p (probabilidad de acción aleatoria del cazador):

valor = (1 - p) × min(hijos) + p × media(hijos)

Esta fórmula modela exactamente el comportamiento del MixedHunterAgent del juego real. Los casos límite son:

- p = 0 → la fórmula reduce a min(hijos) = Minimax puro.
- p = 1 → la fórmula reduce a media(hijos) = Expectimax puro uniforme.
- 0 < p < 1 → interpolación convexa entre peor caso y valor esperado.

Resultados comparando Minimax vs Expectimax (layout swamp_run, 10 partidas, depth=1):

Agente	p=0.0	p=0.3	p=1.0
MinimaxAgent	Variable — asume peor caso siempre	Subóptimo — ignora aleatoriedad	Muy conservador
ExpectimaxAgent	Equivalente a Minimax	Mejor score esperado	Estrategia más agresiva y correcta

Limitación de Expectimax: si p es pequeño pero el cazador tiene un comportamiento catastrófico (captura en un turno), Expectimax puede subestimar el riesgo real porque promedia con los casos benignos. En estos escenarios, Minimax ofrece mayor seguridad.

7 AlphaBetaAgent (Bono)

Complejidad en Tiempo: O(b^(d×n/2)) en el mejor caso (orden óptimo de acciones). O(b^(d×n)) en el peor caso (orden inverso).

Complejidad en Espacio: O(d×n) — igual que Minimax.

¿Es completo?: Sí. Produce el mismo resultado que Minimax, solo más eficientemente.

¿Es óptimo?: Sí. Alpha-Beta es una optimización de Minimax que no altera la decisión final. Solo elimina ramas que no pueden influir en el resultado.

Alpha-Beta mantiene dos valores: α (mejor garantizado para MAX) y β (mejor garantizado para MIN). Se poda con desigualdad estricta:

- Nodo MAX: si valor > β → podar. MAX nunca elegirá esta rama porque MIN ya tiene algo mejor.
- Nodo MIN: si valor < α → podar. MIN nunca elegirá esta rama porque MAX ya tiene algo mejor.

Decisión crítica — desigualdad estricta: se poda SOLO cuando valor > β o valor < α (sin igualdad). Podar en igualdad puede eliminar acciones con el mismo valor óptimo, alterando potencialmente la acción elegida sin cambiar su valor.

Beneficio práctico: permite explorar el doble de profundidad en el mismo tiempo que Minimax en el mejor caso. En layouts grandes como pacman_maze o triple_threat, depth=3 con Alpha-Beta puede ser más efectivo que depth=2 con Minimax puro.

<u>Comparación de Agentes Adversariales</u>

Agente	Complejidad Temporal	Cazador p=0	Cazador p>0	Caso ideal
Minimax	O(b^(d×n))	Óptimo	Conservador	Cazador 100% greedy
AlphaBeta (Bono)	O(b^(d×n/2))*	Óptimo	Conservador	Minimax con mayor profundidad
Expectimax	O(b^(d×n))	=Minimax	Óptimo exp.	Cazador mixto o aleatorio

* Mejor caso con orden de acciones favorable.

Nodos adicionales en poda Alfa-Beta: En el peor caso, Alpha-Beta no poda ningún nodo y explora el mismo árbol que Minimax, con complejidad O(b^(d×n)). En el mejor caso, cuando las mejores acciones se evalúan primero, Alpha-Beta reduce la complejidad a aproximadamente O(b^(d×n/2))* explorando un árbol más pequeño que en Minimax.

<u>Pruebas Cuantitativas</u>

Comparación de agentes. Layout: small_hunt (depth = 1)

Agente	Partidas	Victorias	Derrotas	Score promedio
Minimax	10	0	0	-250.0
AlphaBeta (Bono)	10	0	0	-250.0

Expectimax	10	0	0	-250.0

En este layout los tres agentes muestran el mismo comportamiento y obtienen los mismos resultados. Alpha-Beta produce las mismas decisiones que Minimax, como se espera teóricamente.

Expectimax con distintos valores de p. Layout: swamp_run (depth = 1)

Agente	p	Victorias	Score promedio
Minimax	0	0	333.0
Expectimax	0.0	0	333.0
Expectimax	0.3	0	333.0
Expectimax	1.0	0	333.0

Cuando p = 0, Expectimax se comporta de forma equivalente a Minimax. En este layout el cambio en la probabilidad de acciones aleatorias no altera el resultado final, aunque sí modifica el modelo de decisión del agente.

Múltiples cazadores. Layout: jungle_chase (depth = 2)

Agente	Partidas	Victorias	Derrotas	Score promedio
Minimax	10	10	0	988.0
AlphaBeta (Bono)	10	10	0	988.0
Expectimax (p=0.3)	10	9	1	836.1

Minimax y Alpha-Beta obtienen exactamente los mismos resultados, lo que confirma que Alpha-Beta es una optimización que no cambia la decisión final. Expectimax muestra un comportamiento diferente debido a la incorporación del modelo probabilístico de los cazadores.

Layout difícil. Layout: maze_hunt (depth = 2)

Agente	Partidas	Victorias	Derrotas	Score promedio
Minimax	10	0	0	-250.0
AlphaBeta (Bono)	10	0	0	-250.0
Expectimax (p=0.3)	10	0	0	-250.0

En este layout ninguno de los agentes logra completar las entregas dentro del límite de movimientos. Esto sugiere que la profundidad de búsqueda utilizada es insuficiente o que la función de evaluación no logra guiar adecuadamente al dron en este escenario.

Conclusiones

- En CSP, la clave es la propagación de restricciones. FC y AC-3 no cambian la solución sino el costo de encontrarla. MRV+LCV guían la búsqueda hacia las asignaciones más críticas primero.
- En búsqueda adversaria, el modelo de los oponentes determina la estrategia del dron. Minimax es adecuado para cazadores greedy; Expectimax da mejores estrategias cuando los cazadores tienen comportamiento aleatorio real.
- La función de evaluación es el componente más sensible al contexto. La combinación de señales de progreso y seguridad con la bonificación de urgencia logra un equilibrio razonable, aunque el manejo de ciclos puede mejorarse.